INDIAN INSTITUTE OF TECHNOLOGY ROORKEE
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Explainable Graph Neural Networks for Brain Biomarker Discovery: Mathematics and Logic of the Pipeline

Companion Reference Document to Presentation 2 (Pipeline Execution & Results)

Submitted by:
Animesh Basak
Enrolment No: 26911001

November 2026
Headless NVIDIA Tesla V100 GPU Cluster Setup

Table of Contents

How to read this document

Global notation

Part I: The Four Base Papers

1. Kernel SVM on vectorised connectomes (Abraham et al. 2017)

1.1 Method

1.2 Why it struggles here

2. Random Forest on Graphlet Degree Vectors (Pržulj 2007)

2.1 Graphlets and orbits

2.2 Random Forest

2.3 Limitations

3. node2vec embeddings (Grover & Leskovec 2016)

3.1 Biased second-order random walk

3.2 Skip-gram objective with negative sampling

3.3 Limitations

4. Graph Attention Networks, GATv1 (Veličković et al. 2018)

4.1 Layer

4.2 Static attention (Brody, Alon & Yahav 2022)

4.3 GATv2 fix

Part II: Problem Statement

Part III: The Pipeline, Step by Step

5. Step 1: Data collection

5.1 Why synthetic, and why that is useful

5.2 Base connectivity

5.3 Planted disease signature

5.4 Scanner noise

6. Step 2: Cleaning and preprocessing

6.1 Global signal removal

6.2 Z-scoring

6.3 Sparsification

6.4 Graph objects and storage

6.5 Split

7. Step 3: Exploratory data analysis

7.1 Density and weights
7.2 Locating the signal
7.3 What EDA tells the modeller
8. Step 4: Feature engineering and selection
8.1 The five descriptors
8.2 The scale-invariance problem
8.3 Edge feature
9. Step 5: Model selection
9.1 Input projection
9.2 Edge-weighted GCN layers (×3)
9.3 GATv2 layer with edge attributes (×1)
9.4 Dual readout
9.5 SwiGLU head
9.6 Output probability
10. Step 6: Training
10.1 Loss and its gradient
10.2 Optimiser and schedule
10.3 Mixed precision on the V100
10.4 Early stopping and what the curves say
11. Step 7: Evaluation and tuning
11.1 Metrics
11.2 Results on the 75 test subjects
11.3 The calibration error and the fix
12. Step 8: Explainability with GNNExplainer
12.1 Objective as implemented
12.2 Gradient with respect to the masks
12.3 Extracting biomarkers
12.4 What the explanations actually show
12.5 Why: faithfulness is not truth
Part IV: Interpretation
13. Why the GNN reached 0.746 and not 1.000
13.1 The model cannot see which nodes share a lobe
13.2 How it still got 0.746
13.3 Why the flattened SVM did worse
13.4 Why one scalar wins
14. Comparison with the four base papers

15. What changes before the next review

16. References

How to read this document

The document has four parts. Part I covers the four base papers from the first presentation: what each one computes, the mathematics, and where it breaks for brain graphs. Part II states our problem formally. Part III walks through the pipeline as implemented in the repository, one step at a time, with the derivations behind each step. Part IV interprets the results, including two places where our own earlier claims did not survive a re-check.

Every equation is followed by a "where" list that explains each symbol. If a symbol was explained earlier and keeps the same meaning, it is not repeated. The notation table below lists the symbols used throughout.

To convert to PDF: `pandoc COMPANION_DOCUMENT.md -o companion.pdf --pdf-engine=xelatex -V geometry:margin=2.2cm`.

Global notation

Symbol	Meaning
k	index of a subject (one brain scan, one graph); $k = 1, \dots, N$
N	number of subjects; $N = 500$ in our cohort
n	number of brain regions (graph nodes); $n = 116$ for the AAL-116 atlas
i, j, l, h	indices of brain regions, each in $\{1, \dots, n\}$
$G_k = (V, E_k, X_k)$	graph of subject k: node set V, edge set E_k, node-feature matrix X_k
$R \in$ ¿	functional connectivity (Pearson correlation) matrix
z_{ij}	z-scored correlation between regions i and j
$W \in R^{n \times n}$	sparsified, signed edge-weight matrix used to build the graph
$w_{ij} = \lvert W_{ij} \rvert$	unsigned edge strength
A	binary adjacency, $A_{ij} = 1[W_{ij} \neq 0]$
$N(i)$	neighbours of node i: $\{j : A_{ij} = 1\}$

$k_i = \lvert N(i) \rvert$	(binary) degree of node i
$x_i \in R^5$	input feature vector of node i
$h_i^{(l)} \in R^d$	hidden representation of node i after layer l
d	hidden width; $d = 192$
$y_k \in \{0, 1\}$	true label: 0 Healthy, 1 Diseased
¿ p_k	model probability that subject k is Diseased
θ	all trainable parameters of the model
$1[\cdot]$	indicator: 1 if the condition holds, 0 otherwise
$\cdot$	element-wise (Hadamard) product
¿ ∨ ¿	vector concatenation
$\sigma(\cdot)$	logistic sigmoid $1/(1+e^{-x})$

Part I: The Four Base Papers

1. Kernel SVM on vectorised connectomes (Abraham et al., NeuroImage 2017)

1.1 Method

The connectivity matrix is symmetric, so only the upper triangle carries information. The method reads that triangle into a vector:

$$v_k = vec_{i<j}(W^{(k)}) \in R^p, p = \frac{n(n-1)}{2} = \frac{116 \cdot 115}{2} = 6670$$

Where: v_k is the feature vector of subject k; $vec_{i<j}$ lists the entries W_{ij} with $i<j$ in a fixed order; p is the number of features.

A support vector machine looks for the separating hyperplane with the widest margin. In its dual form with a kernel it solves

$$max_\alpha \sum_{k=1}^{N_{tr}} \alpha_k - \frac{1}{2} \sum_{k=1}^{N_{tr}} \sum_{m=1}^{N_{tr}} \alpha_k \alpha_m s_k s_m K(v_k, v_m) \text{ s.t. } 0 \le \alpha_k \le C, \quad \sum_k \alpha_k s_k = 0$$

Where: N_{tr} is the number of training subjects (350 here); $\alpha_k \geq 0$ is the dual variable (Lagrange multiplier) of subject k, and subjects with $\alpha_k > 0$ are the support vectors; $s_k = 2y_k - 1 \in \{-1, +1\}$ is the label re-coded to ± 1; $C > 0$ is the box constraint that limits how much any single subject can push the boundary (we used $C = 1$); K is the kernel.

The RBF kernel is

$$K(v_k, v_m) = \exp\left(-\gamma \vee ¿ \, v_k - v_m \vee ¿_2^2\right)$$

Where: $\gamma > 0$ sets the kernel width (scikit-learn's `"scale"` uses $\gamma = 1/(p \cdot Var(v))$); $¿ \vee \cdot \vee ¿_2$ is the Euclidean norm.

The decision function for a new subject v is $f(v) = \sum_k^{\square} \alpha_k s_k K(v_k, v) + b$, with b the bias recovered from any support vector on the margin. The predicted class is $sign(f(v))$.

1.2 Why it struggles here

1. Dimension against sample size. $p = 6670$ features from $N_{tr} = 350$ subjects. For the RBF kernel the distance $¿ \vee v_k - v_m \vee ¿_2^2 = \sum_{q=1}^{p} (v_{kq} - v_{mq})^2$ is a sum of 6,670 terms. In our data most of those terms are pure subject-specific noise. By the law of large numbers the sum concentrates around the same value for every pair, so all subjects look roughly equidistant and the kernel matrix tends towards a constant plus a diagonal. The class signal is a 30% magnitude change on about 1,066 intra-lobe entries, and it gets drowned.

2. No topology. Permuting the 6,670 coordinates changes nothing for the SVM, so it cannot use the fact that some edges share an endpoint or a lobe.

3. Sign problem. Our signal is in $|W_{ij}|$: diseased intra-lobe correlations grow in magnitude whether they are positive or negative. The signed values W_{ij} have the same mean in both classes. A linear model on signed values therefore has nothing to latch onto, and the RBF kernel only sees the signal through second-order distance effects.

Our re-run on the identical split: test AUC 0.530, accuracy 53.3%, macro-F1 0.533. An L2 logistic regression on the same vector gets AUC 0.509. Both are at chance. The README figure of 0.671 did not reproduce.

2. Random Forest on Graphlet Degree Vectors (Pržulj, Bioinformatics 2007)

2.1 Graphlets and orbits

A graphlet is a small connected induced subgraph, here on 2 to 5 nodes. Up to isomorphism there are 30 such graphlets and, inside them, 73 automorphism orbits (positions a node can occupy). The graphlet degree vector of node u counts how often u touches each orbit:

$$GDV(u)=¿$$

Where: orbit 0 is "endpoint of an edge", so $u_0=k_u$ is the degree; orbit 3 is "corner of a triangle", so u_3 counts triangles through u; higher orbits encode larger motifs.

The repo implementation keeps 5 orbits and summarises each graph with three statistics:

$$\Phi(G)=\left[mean_u GDV(u),\ std_u GDV(u),\ max_u GDV(u)\right]\in R^{15}$$

2.2 Random Forest

Each of T trees is grown on a bootstrap resample. At every split a random subset of features is tried, and the threshold that most reduces Gini impurity is kept:

$$Gini(S)=1-\sum_{c\in\{0,1\}}^{\square}\pi_c(S)^2,\Delta=Gini(S)-\frac{|S_L|}{|S|}Gini(S_L)-\frac{|S_R|}{|S|}Gini(S_R)$$

Where: S is the set of training subjects reaching the node; $\pi_c(S)$ is the fraction of class c in S; S_L,S_R are the left and right children after the split; Δ is the impurity decrease. The forest predicts $\overset{¿}{p}(G)=\frac{1}{T}\sum_{b=1}^{T}\overset{¿}{p_b}(\Phi(G))$, the average of the tree probabilities $\overset{¿}{p_b}$.

2.3 Limitations

- Graphlet counts are **binary-topology** quantities. With a 76% dense graph almost every triple is a triangle, so counts saturate and barely differ between subjects. Edge magnitude, which is where our signal lives, is thrown away.
- Features are fixed before training. No gradient flows back to decide which motif or which edge matters.
- Mean/std/max over nodes discards region identity, which is the same blind spot our GNN has (Section 10.3).

The README lists AUC 0.692 for this baseline, but there is no run log, so we mark it unverified.

3. node2vec embeddings (Grover & Leskovec, KDD 2016)

3.1 Biased second-order random walk

A walk that has just moved from t to v chooses the next node x with probability

$$P(c_{r+1}=x \mid c_r=v, c_{r-1}=t)=\frac{\alpha_{pq}(t,x)\,w_{vx}}{Z_v}, \alpha_{pq}(t,x)=\begin{cases}1/p & d_{tx}=0\\ 1 & d_{tx}=1\\ 1/q & d_{tx}=2\end{cases}$$

Where: c_r is the node visited at step r of the walk; w_{vx} is the edge weight; Z_v normalises the probabilities to sum to one; d_{tx} is the shortest-path distance between the previous node t and the candidate x (0 means "go back to t", 1 means "stay near t", 2 means "move away"); p is the return parameter (large p discourages going back); q is the in-out parameter (q<1 favours exploring outward, DFS-like; q>1 keeps the walk local, BFS-like).

3.2 Skip-gram objective with negative sampling

Each node gets a vector $\phi(u) \in R^D$. Nodes that co-occur within a window of the walks should have similar vectors:

$$max_\phi \sum_{u \in V} \sum_{c \in C(u)} \left[\log \sigma\left(\phi(c)^\top \phi(u)\right)+\sum_{m=1}^{M} E_{c' \sim P_n} \log \sigma\left(-\phi(c')^\top \phi(u)\right)\right]$$

Where: $C(u)$ is the multiset of context nodes seen within the window around u across all walks; D is the embedding size (64 in the repo); M is the number of negative samples; P_n is the noise distribution, usually degree$^{3/4}$; c' is a negative (random) node.

The gradient with respect to $\phi(u)$ for one positive pair is $\left(1-\sigma\left(\phi(c)^\top \phi(u)\right)\right)\phi(c)$. It pulls $\phi(u)$ towards $\phi(c)$, and the negatives push it away from random nodes.

A graph-level vector is then $\bar{\phi}(G_k)=\frac{1}{n}\sum_u \phi_k(u)$, which is fed to a classifier.

3.3 Limitations

- **Unsupervised.** The walks never see y_k, so the embedding optimises co-occurrence, not disease separation.
- **Not comparable across subjects.** Each subject's embedding is trained separately. The skip-gram objective depends only on inner products, so any rotation $Q\phi$ with $Q^\top Q=I$ is an equally good solution. Averaging the vectors and comparing them across subjects then compares arbitrarily rotated spaces.
- **Mean pooling** removes most of what remains.

- In the repo, when the `node2vec` package is missing, the code silently replaces the walks with a spectral embedding. Log which path ran before quoting the number (README: AUC 0.635, unverified).

4. Graph Attention Networks, GATv1 (Veličković et al., ICLR 2018)

4.1 Layer

$$e_{ij} = LeakyReLU\left(a^{\top}\left[Wh_i \vee ¿ Wh_j\right]\right), \alpha_{ij} = \frac{\exp\left(e_{ij}\right)}{\sum_{l \in N(i)} \exp\left(e_{il}\right)}, h_i^{'} = \sigma\left(\sum_{j \in N(i)} \alpha_{ij} W h_j\right)$$

Where: $W \in R^{d^{'} \times d}$ is a shared linear map; $a \in R^{2d^{'}}$ is the attention vector; e_{ij} is the unnormalised score of neighbour j for query i; α_{ij} are the softmax-normalised attention weights over the neighbours of i; LeakyReLU is $max\left(x, 0.2x\right)$.

4.2 Static attention (Brody, Alon & Yahav, ICLR 2022)

Split $a = [a_1; a_2]$ with $a_1, a_2 \in R^{d^{'}}$. Then

$$a^{\top}\left[Wh_i \vee ¿ Wh_j\right] = a_1^{\top} Wh_i + a_2^{\top} Wh_j$$

LeakyReLU is monotone increasing, and the softmax over j ignores the term $a_1^{\top} Wh_i$ because it is constant in j. So, for every query i, the ranking of neighbours is decided by $a_2^{\top} Wh_j$ alone:

$$¿ max_{j \in N(i)} \alpha_{ij} = ¿ max_{j \in N(i)} a_2^{\top} Wh_j$$

The same node "wins" for every query that has it as a neighbour. This is what Brody et al. call **static attention**: the function cannot express "region i should attend to j, but region $i^{'}$ should attend to $j^{'}$" whenever j and $j^{'}$ are both available to both. On a 76%-dense brain graph most nodes share most neighbours, so the restriction bites hard.

(Strictly, the nonlinearity can change the *weights*, not the *order*, so "rank collapse" in deck 1 should be read as "ranking is the same for all queries".)

4.3 GATv2 fix

Move the nonlinearity inside:

$$e_{ij} = a^{\top} LeakyReLU\left(W_s h_i + W_t h_j\right)$$

Now h_i and h_j interact through the nonlinearity before the dot product with a, so the ranking over j can depend on i. Brody et al. prove that GATv2 is a universal approximator

of attention functions, while GATv1 is not. This is the layer we use, with an extra edge term (Section 9.3).

Part II: Problem Statement

For each subject k we observe a correlation matrix $R^{(k)}$ and a label y_k. We want:

1. A classifier f_θ such that

$$\hat{p}_k = P(y_k = 1 \mid G_k) = softmax\big(f_\theta(A_k, W_k, X_k)\big)_1$$

which is **permutation invariant**: for any permutation matrix Π, $f_\theta(\Pi A \Pi^\top, \Pi W \Pi^\top, \Pi X) = f_\theta(A, W, X)$. A permutation matrix has exactly one 1 in every row and column; multiplying by it re-orders the nodes. Invariance means the output does not depend on the arbitrary order in which regions are listed.

2. An explanation for each prediction: a small edge subset $E_s \subseteq E_k$ such that the model's prediction from $G_s = (V, E_s)$ alone stays close to its prediction from the full graph. GNNExplainer formalises this as maximising mutual information:

$$\max_{G_s} \; MI(Y, G_s) = H(Y) - H(Y \mid G = G_s)$$

Where: Y is the model's predicted label as a random variable; $H(Y)$ is its entropy, fixed for a trained model; $H(Y \mid G = G_s)$ is the entropy of the prediction when only G_s is shown. Maximising MI is the same as making the prediction on G_s as certain as possible.

3. No leakage: the sets of training and test subjects are disjoint, $S_{tr} \cap S_{te} = \varnothing$, and nothing computed from test subjects (thresholds included) is used to fit anything.

Point 3 matters for Section 11: the repo broke it once, in the threshold calibration.

Part III: The Pipeline, Step by Step

5. Step 1: Data collection

5.1 Why synthetic, and why that is useful

Real cohorts (ADNI, PPMI) need data-use agreements. Until they arrive, the pipeline runs on a synthetic cohort generated by `data/synthetic/generate_synthetic.py` and called from `scripts/train_launcher.py`. The synthetic data has one big advantage: **we know the true biomarker**, so we can measure whether the explanations recover it. On real data that check cannot be made.

Cohort: $N=500$, labels alternate with the index (250 per class), sites assigned as site$=i \bmod 3$ (167/167/166), ages uniform in 50–84. Since labels follow $i \bmod 2$ and sites follow $i \bmod 3$, every (site, label) cell is balanced and the site cannot act as a confound.

5.2 Base connectivity

$$\Sigma = BB^{\top} + nI, B \in R^{n \times n},\ B_{ij} \overset{iid}{\sim} N(0,1), R_{ij} = \frac{\Sigma_{ij}}{\sqrt{\square}}$$

Where: B is a random Gaussian matrix; $BB^{\top}$ is symmetric positive semi-definite; adding nI (the identity scaled by n) makes it strictly positive definite, so Σ is a valid covariance matrix; dividing by the square roots of the diagonal turns covariance into correlation, so $R_{ii}=1$.

Size of the off-diagonal entries: $\Sigma_{ij} = \sum_{l}^{\square} B_{il} B_{jl}$ is a sum of n products of independent standard normals, with mean 0 and variance n. The diagonal is $\Sigma_{ii} = \sum_{l}^{\square} B_{il}^2 + n \approx 2n$. So

$$SD(R_{ij}) \approx \frac{\text{¿}\frac{1}{2^{\text{¿}\frac{1}{2^{-0.046}\sqrt{\square}}}\sqrt{\square}}\sqrt{\square}}{\square}$$

Healthy correlations are tiny and centred on zero.

5.3 Planted disease signature

For Diseased subjects only:

$$R_{i,i+1} \leftarrow 0.4\, R_{i,i+1} \text{ for } i \in \{0,2,\ldots,88\} \quad (45 \text{ homotopic left–right pairs})$$

$$R_{ij} \leftarrow min(1, 1.3\, R_{ij}) \text{ for } i \neq j \text{ in the same lobe}$$

Where: AAL lists left and right homologues next to each other (e.g. index 0 `Precentral_L`, index 1 `Precentral_R`), so $(i, i+1)$ with even $i<90$ is a homotopic pair; the seven lobes are frontal (0–27), limbic (28–41), occipital (42–55), parietal (56–69), subcortical (70–79), temporal (80–89) and cerebellar (90–115).

Biologically this mimics two reported patterns: weaker inter-hemispheric coupling and stronger, more segregated local coupling. Mathematically, note that multiplying by 1.3 scales the magnitude and keeps the sign. The **mean** of R_{ij} stays zero in both classes. Only $E|R_{ij}|$ changes. That single fact explains most of the results.

The lobe assignment overlaps with the homotopic pairs. Pairs like (Precentral_L, Precentral_R) are in the same lobe, so they are scaled by 1.3 and by 0.4, a net factor of 0.52.

5.4 Scanner noise

$$R \leftarrow clip\left(R+\frac{1}{2}(\varepsilon+\varepsilon^{\top}),-1,1\right), \varepsilon_{ij} \overset{iid}{\sim} N(0,\sigma_s^2), \quad \sigma_s \in \{0.02,0.04,0.06\}$$

Where: ε is a noise matrix; averaging it with its transpose keeps R symmetric; its diagonal is zeroed; σ_s depends on the site. The noise SD per entry is $\sigma_s/\sqrt{\square}$, i.e. 0.014–0.042, which is the same order as the 0.046 signal SD. The classifier has to see through noise as large as the effect.

6. Step 2: Cleaning and preprocessing

Implemented in `neo4j_layer/etl/transform.py` and called in `step_etl`.

6.1 Global signal removal

$$\mu_g = \frac{1}{n(n-1)} \sum_{i \neq j} R_{ij}, R_{ij} = R_{ij} - \mu_g \ (i \neq j), R_{ii} = 1$$

Where: μ_g is the mean off-diagonal correlation of that subject, a proxy for the global BOLD signal; subtracting it removes a subject-wide shift, which in real fMRI comes from breathing, heart rate and head motion. Real global signal regression works on time series, not on correlation matrices. This is a matrix-level stand-in and should be described that way.

6.2 Z-scoring

$$\mu = \frac{1}{n(n-1)} \sum_{i \neq j} R_{ij}, \sigma^2 = \frac{1}{n(n-1)} \sum_{i \neq j} (R_{ij} - \mu)^2, z_{ij} = \frac{R_{ij} - \mu}{\sigma}, z_{ii} = 0$$

Where: μ and σ are computed **within one subject**. After this, each subject's off-diagonal entries have mean 0 and SD 1. This removes site-level scale differences (good), and it also changes the disease signal from an absolute magnitude to a *relative* one: diseased intra-lobe entries are larger **relative to that subject's own inter-lobe entries** (Fig 2 shows inter-lobe $|z|$ going *down* in Diseased subjects, which is purely this renormalisation).

6.3 Sparsification

$$W_{ij} = z_{ij} \cdot 1[|z_{ij}| \geq \tau], \tau = 0.3$$

Where: τ is the pruning threshold. Since z is roughly standard normal, $P(|z|<0.3) = 2\Phi(0.3) - 1 \approx 0.236$, where Φ is the standard normal CDF. So about 76.4% of edges survive. The measured value is $5,092 \pm 34$ of 6,670 (76.3%), which matches.

Deck 1 wrote this as $|r_{ij}| \geq 0.3$. On raw correlations of SD 0.046 that threshold would delete almost every edge. The code applies it to z, and the slides now say so.

6.4 Graph objects and storage

The upper triangle with $W_{ij} \neq 0$ is written to `edge_list.csv` (2,545,839 rows for 500 subjects) and can be loaded into Neo4j as `(:BrainRegion)-[:CONNECTED_TO {subjectId, weight}]->(:BrainRegion)`, with one shared set of 116 region nodes and subject-tagged relationships. For PyTorch Geometric each undirected edge becomes two directed entries in `edge_index` $\in Z^{2 \times 2|E|}$, with the signed z_{ij} in `edge_attr` $\in R^{2|E| \times 1}$.

6.5 Split

`subject_level_split` draws a stratified 70/15/15 split with seed 42: 350 train, 75 validation, 75 test. Indices are mapped back through `PatientID`, so a subject's graph can only land in one split. We verified with an explicit assertion that the train and test sets share no subject.

7. Step 3: Exploratory data analysis

7.1 Density and weights

Edges per subject: $5,092 \pm 34$; the class distributions overlap almost completely (Fig 1). Weights are symmetric around zero with SD 1.14 and exactly half negative. Nothing in the first-order statistics separates the classes.

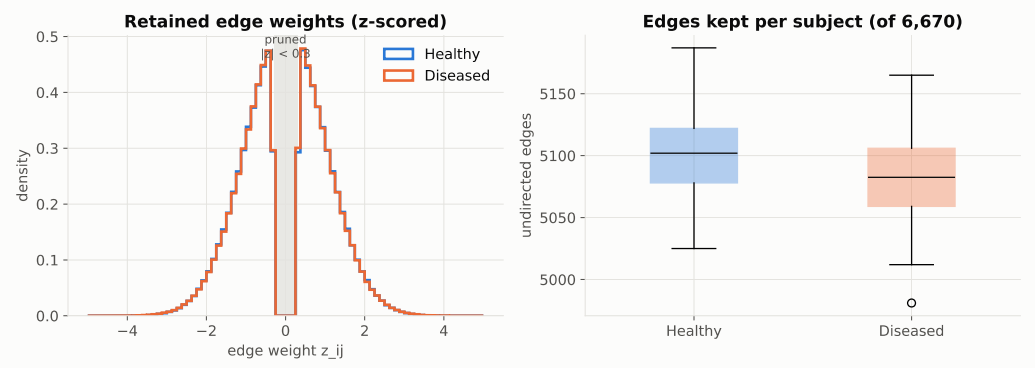


Figure 1: Connectome edge density and weight distributions across healthy and diseased cohorts (left: edge weight probability density; right: retained edge count per subject at $|z| \geq 0.3$).

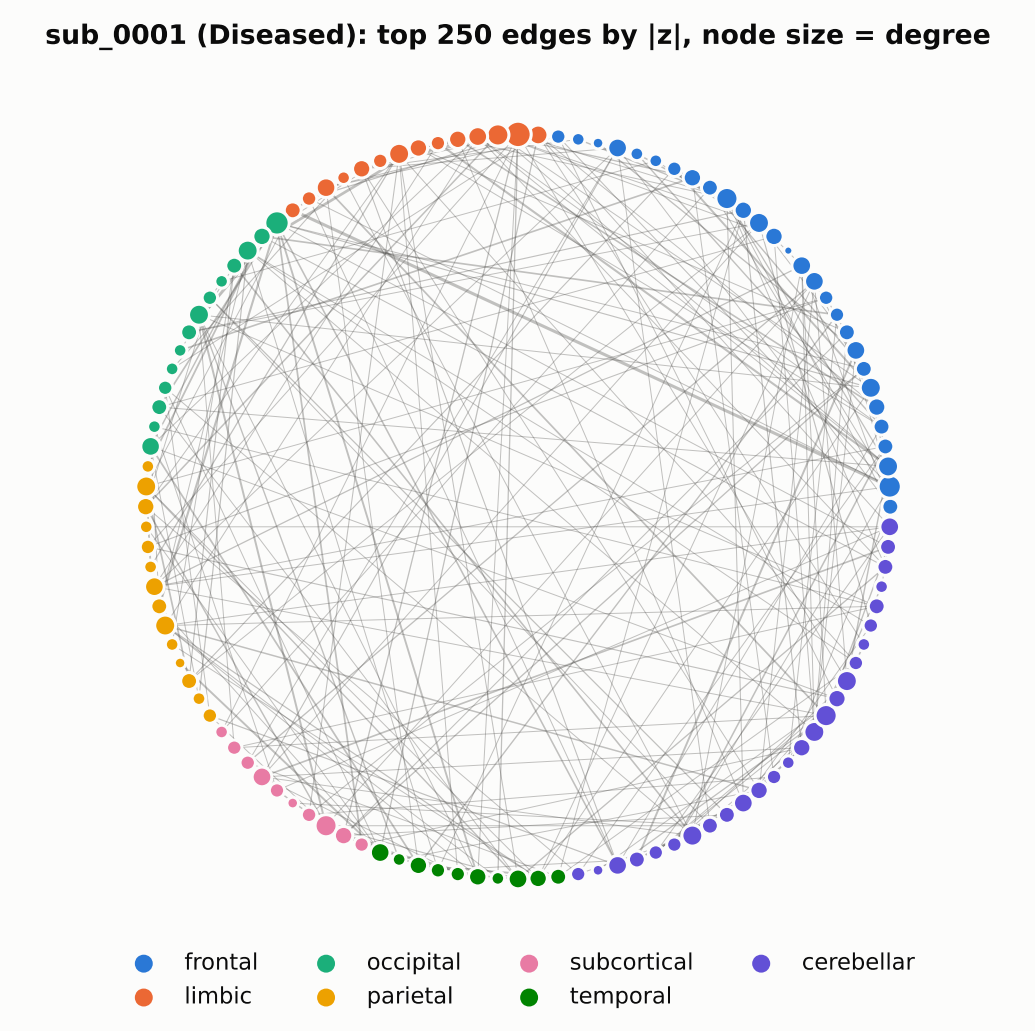


Figure 4: Single-subject functional brain connectome graph on the AAL-116 atlas, illustrating modular lobe organisation and anatomical connectivity topology.

7.2 Locating the signal

Define the group-difference matrix

$$\Delta_{ij} = \frac{1}{|D|}\sum_{k \in D}^{\square} \left|W_{ij}^{(k)}\right| - \frac{1}{|H|}\sum_{k \in H}^{\square} \left|W_{ij}^{(k)}\right|$$

Where: D and H are the sets of Diseased and Healthy subjects. Fig 3 plots Δ with the regions ordered by lobe. Every diagonal block is red ($\Delta > 0$) and the off-block area is faintly blue. Inside the blocks a darker blue line runs just off the diagonal: those are the homotopic pairs, net-scaled by 0.52.

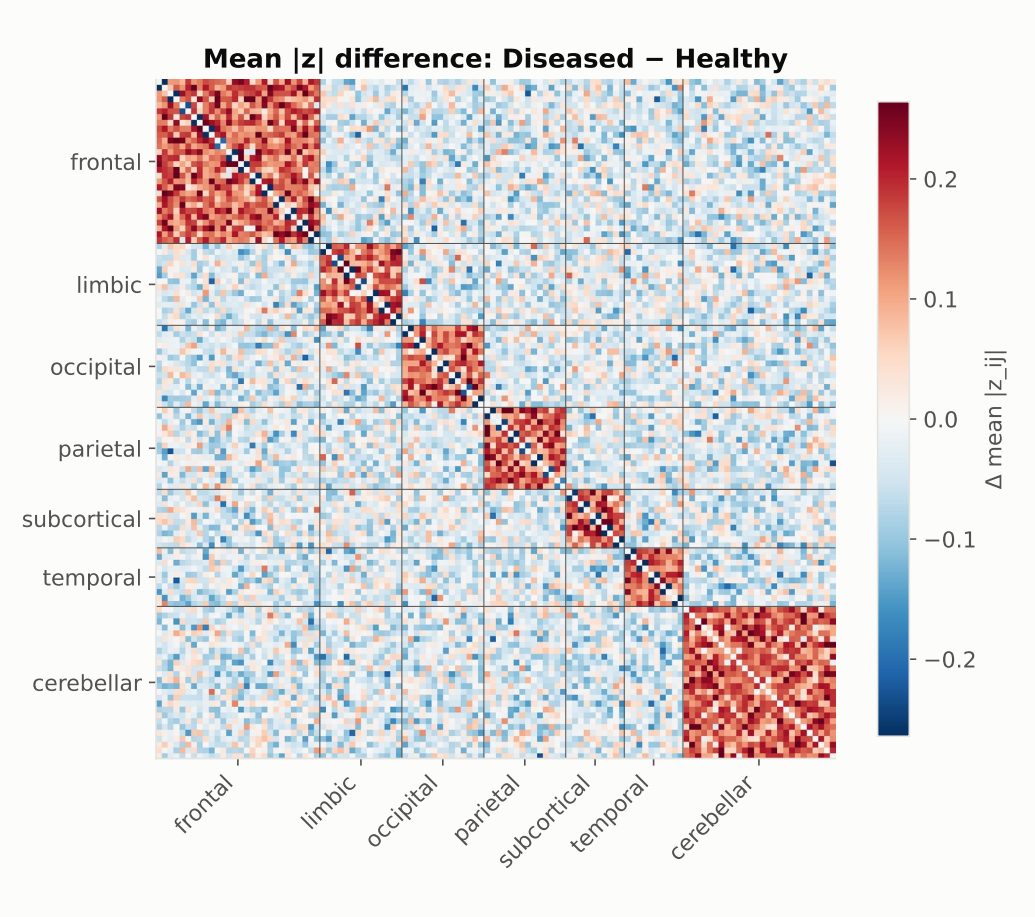


Figure 3: Group-difference matrix Δ_ij = mean(|W_ij|_D) - mean(|W_ij|_H) ordered by anatomical lobes, displaying distinct intra-lobe elevation blocks and attenuated homotopic L–R pairs.

Now take one number per subject:

$$s_k = \frac{1}{|I|} \sum_{(i,j) \in I}^{\square} \left| W_{ij}^{(k)} \right|, I = \{(i,j) : i < j,\ lobe(i) = lobe(j)\}$$

Where: I is the set of intra-lobe pairs, 1,066 pairs in total (16.0% of all 6,670).

s_k alone separates the 75 test subjects with **AUC 1.000**. Healthy medians sit near 0.765 and Diseased near 0.895 with no overlap (Fig 2).

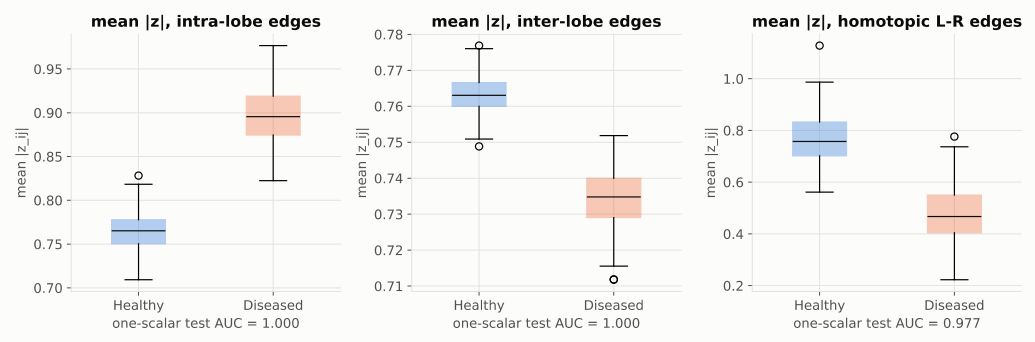


Figure 2: Separation of healthy and diseased test subjects using a single scalar feature (mean intra-lobe correlation strength), demonstrating an empirical oracle AUC of 1.000.

7.3 What EDA tells the modeller

The task is easy **if** the model knows which pairs are intra-lobe, and hard if it does not. A flattened vector does in principle carry pair identity (coordinate q is always the same pair), but it holds signed values, and the signal is in $|\cdot|$, spread thinly across 1,066 coordinates. A message-passing GNN sees magnitudes but, as Section 10.3 shows, it does not see region identity. Both baselines and our model are therefore handicapped in different ways.

8. Step 4: Feature engineering and selection

Node features are computed in `models/gnn/dataset.py` with NetworkX on the graph whose edge weights are $w_{ij} = |W_{ij}|$.

8.1 The five descriptors

Normalised degree

$$x_i^{(1)} = \frac{k_i}{max_j k_j}$$

Where: k_i counts neighbours, ignoring weights.

PageRank: the stationary distribution of a random surfer who follows weighted edges with probability δ and teleports uniformly otherwise:

$$\pi = \frac{1-\delta}{n} 1 + \delta P^{\top} \pi, P_{ij} = \frac{w_{ij}}{\sum_l w_{il}}, x_i^{(2)} = \pi_i$$

Where: $\pi \in R^n$ with $\sum_i^{\square} \pi_i = 1$; $\delta = 0.85$ is the damping factor; P is the row-stochastic transition matrix; 1 is the all-ones vector. It is computed by power iteration $\pi^{(t+1)} = \frac{1-\delta}{n} 1 + \delta P^{\top} \pi^{(t)}$, which converges because the map is a contraction with factor $\delta < 1$.

Weighted clustering (Onnela et al. 2005):

$$x_i^{(3)} = C_i = \frac{1}{k_i(k_i-1)} \sum_{j,h \in N(i)}^{\square} \left(\tilde{w}_{ij} \tilde{w}_{ih} \tilde{w}_{jh} \right)^{1/3}, \tilde{w}_{ij} = \frac{w_{ij}}{max_{(a,b)} w_{ab}}$$

Where: the sum runs over ordered pairs of neighbours that are themselves connected; $\tilde{w}$ is the weight scaled by the largest weight in that graph; the cube root is the geometric mean of the triangle's three edges.

Betweenness

$$x_i^{(4)} = b_i = \frac{2}{(n-1)(n-2)} \sum_{s \neq i \neq t}^{\square} \frac{\sigma_{st}(i)}{\sigma_{st}}$$

Where: σ_{st} is the number of shortest paths from s to t; $\sigma_{st}(i)$ is the number of those passing through i; the prefactor normalises to [0,1] for an undirected graph. NetworkX computes path length as the **sum of the `weight` attribute**, so a strong correlation counts as a long hop. A distance such as $1/w_{ij}$ or $-\log w_{ij}$ is what was intended.

Eigenvector centrality

$$w e = \lambda_1 e, x_i^{(5)} = \frac{e_i}{max_j e_j}$$

Where: w here is the weighted adjacency matrix; λ_1 is its largest eigenvalue; e is the corresponding eigenvector, which is non-negative by the Perron–Frobenius theorem because $w \geq 0$ and the graph is connected. Node i is central if its neighbours are central.

8.2 The scale-invariance problem

Suppose every weight in a lobe is multiplied by a constant c. Degree does not change, since it counts edges and the threshold was applied on z. PageRank changes only through ratios $w_{ij} / \sum_l^{\square} w_{il}$, and a node whose edges are mostly within its lobe sees those ratios move only slightly. Clustering is normalised by the global maximum weight. Eigenvector centrality is normalised by its maximum. Each feature is either invariant or nearly invariant to the kind of change the disease makes.

The feature set was picked for interpretability: hubs, bridges and cliques are vocabulary neuroscientists use. It was not picked for sensitivity to the effect, and the EDA shows the effect would have been caught by a feature as plain as "mean $|w|$ to same-lobe neighbours". Feature *selection* was never run (no ablation over the five), which is an obvious next step.

8.3 Edge feature

The edge attribute z_{ij} is the only input that carries magnitude directly. It enters the GCN normalisation as $w_{ij}=|z_{ij}|$ and the GATv2 score as the signed z_{ij}. If anything in the model learns the disease, it learns it through this channel.

9. Step 5: Model selection

The full model (`models/gnn/gcn_gat_model.py`) has 521,378 trainable parameters at $d=192$. We go through it layer by layer, with shapes, and derive the backward pass where it is useful.

9.1 Input projection

$$h_i^{(0)}=Drop_{0.25}\left(SiLU\left(W_{in}x_i+b_{in}\right)\right), W_{in}\in R^{192\times 5},\ b_{in}\in R^{192}$$

Where: $SiLU(u)=u\sigma(u)$, also called Swish; $Drop_{0.25}$ zeroes each coordinate with probability 0.25 during training and divides the survivors by 0.75, so the expected value is unchanged.

SiLU derivative: $\frac{d}{du}u\sigma(u)=\sigma(u)+u\sigma(u)(1-\sigma(u))=\sigma(u)\left(1+u(1-\sigma(u))\right)$. Unlike ReLU it is smooth and non-zero for negative u, so no unit can die permanently.

9.2 Edge-weighted GCN layers (×3)

PyG's `GCNConv` with `edge_weight` computes, for all nodes at once,

$$\hat{A}=D^{-1/2}WD^{-1/2}, W=w+I, D_{ii}=\sum_j W_{ij}=1+\sum_j w_{ij}$$

$$M^{(l)}=\hat{A}H^{(l)}\Theta^{(l)}+1b^{(l)\top}, H^{(l+1)}=H^{(l)}+Drop\left(SiLU\left(BN\left(M^{(l)}\right)\right)\right)$$

Where: $H^{(l)}\in R^{n\times d}$ stacks the $h_i^{(l)}$ as rows; W is the weighted adjacency plus self-loops of weight 1 (each node keeps part of its own state); D is the diagonal weighted-degree matrix; $\hat{A}$ is the symmetrically normalised propagation matrix, with entry $\hat{A}_{ij}=W_{ij}/\sqrt{\square}$;

$\Theta^{(l)} \in R^{d \times d}$ and $b^{(l)} \in R^{d}$ are the layer weights; BN is batch normalisation (below); the leading $H^{(l)}+¿$ is the residual connection.

Per node this is

$$m_i^{(l)} = \sum_{j \in N(i) \cup \{i\}} \frac{W_{ij}}{\Theta^{(l)\top} h_j^{(l)} + b^{(l)} \sqrt{\square}}$$

Link to the message-passing lecture. Drop Θ and the nonlinearity, and use $D^{-1} W$ instead of the symmetric form: the update becomes $h_i \leftarrow \sum_j \frac{W_{ij}}{D_{ii}} h_j$, which is exactly the probabilistic relational classifier $P(Y_v = c) = \frac{1}{\sum_u A_{vu}} \sum_u A_{vu} P(Y_u = c)$. The symmetric normalisation $D^{-1/2} W D^{-1/2}$ is the same diffusion matrix used in Correct & Smooth. Its eigenvalues lie in $(-1, 1¿$ and the top eigenvector is $D^{1/2} 1$. A GCN is relational classification with learned feature maps in between.

Over-smoothing check. Repeated application gives $\hat{A}^{L} H \to u_1 u_1^{\top} H$ with $u_1 \propto D^{1/2} 1$: every node converges to the same vector up to a degree factor. On a 76%-dense graph the second eigenvalue of $\hat{A}$ is small, so mixing is fast. Three layers on such a graph already bring nodes close to the global average. The residual connection $H^{(l)}+\ldots$ slows this down, because the update becomes $H^{(l+1)} \approx (I + \hat{A}\Theta) H^{(l)}$ instead of $\hat{A}\Theta H^{(l)}$.

Backward pass of one GCN layer. Let L be the loss and $G_M = \partial L / \partial M \in R^{n \times d}$ the upstream gradient. Since $M = \hat{A} H \Theta$ (bias omitted):

$$\frac{\partial L}{\partial \Theta} = (\hat{A} H)^{\top} G_M = H^{\top} \hat{A}^{\top} G_M, \quad \frac{\partial L}{\partial H} = \hat{A}^{\top} G_M \Theta^{\top} = \hat{A} G_M \Theta^{\top}$$

Where: the last equality uses $\hat{A}^{\top} = \hat{A}$ (symmetric). The derivation: write $M_{ia} = \sum_{j,b} \hat{A}_{ij} H_{jb} \Theta_{ba}$; then $\partial M_{ia} / \partial \Theta_{ba} = \sum_j \hat{A}_{ij} H_{jb} = (\hat{A} H)_{ib}$, so $\partial L / \partial \Theta_{ba} = \sum_i (G_M)_{ia} (\hat{A} H)_{ib}$, which is $(H^{\top} \hat{A}^{\top} G_M)_{ba}$. The same index bookkeeping gives the gradient for H. The gradient for the inputs is also propagated by $\hat{A}$: errors diffuse along the same edges that messages travel, the same idea as the Correct step of C&S.

The gradient also flows into the edge weights, since $\hat{A}$ depends on w:

$\partial \hat{A}_{ij}/\partial w_{ij} = \frac{1}{-\frac{W_{ij}}{2}\left(D_{ii}^{-3/2}D_{jj}^{-1/2}+D_{ii}^{-1/2}D_{jj}^{-3/2}\right)\sqrt{\square}}$ for $i \neq j$. The inputs w are fixed data during training, but GNNExplainer later differentiates through exactly this expression with respect to its mask (Section 12).

Batch normalisation over the batch of all nodes in all graphs of the mini-batch:

$$BN(m)_a = \gamma_a \frac{m_a - \mu_a^B}{+\beta_a \sqrt{\square}}$$

Where: a indexes the feature channel; μ_a^B, σ_a^B are the mean and SD of channel a over all nodes in the batch (32×116 nodes); γ_a, β_a are learned scale and shift; $\epsilon = 10^{-5}$ avoids division by zero. At test time running averages replace batch statistics.

9.3 GATv2 layer with edge attributes (×1)

PyG's `GATv2Conv(192, 48, heads=4, edge_dim=1, concat=True)`:

$$e_{ij}^{(c)} = a_c^{\top} LeakyReLU_{0.2}\left(W_s^{(c)}h_i + W_t^{(c)}h_j + W_e^{(c)}z_{ij}\right), c = 1,\ldots,4$$

$$\alpha_{ij}^{(c)} = \frac{\exp e_{ij}^{(c)}}{\sum_{l \in N(i)}^{\square} \exp e_{il}^{(c)}}, g_i = \left| \hat{}_{c=1}^{4} \sum_{j \in N(i)}^{\square} \alpha_{ij}^{(c)} W_t^{(c)} h_j + b \right.$$

$$h_i^{(4)} = h_i^{(3)} + Drop\left(SiLU\left(BN(g_i)\right)\right)$$

Where: c indexes the attention head; $W_s^{(c)}, W_t^{(c)} \in R^{48 \times 192}$ are the source (query) and target (key/value) maps; $W_e^{(c)} \in R^{48 \times 1}$ lifts the scalar edge attribute into the head's space; $a_c \in R^{48}$ is the head's attention vector; $\hat{} \vee \hat{}$ concatenates the four 48-dimensional head outputs into 192; $b \in R^{192}$ is a bias; attention dropout of 0.25 is applied to α during training. PyG adds self-loops by default and fills their edge attribute with the mean of the incoming attributes.

Because z_{ij} sits inside the LeakyReLU next to h_i and h_j, the edge sign and magnitude can change both the score and the ranking. This is the main route by which the model can see the disease signal.

Softmax Jacobian (used for attention and for the output):

$$\frac{\partial \alpha_{ij}}{\partial e_{il}} = \alpha_{ij}\left(1[j=l] - \alpha_{il}\right)$$

Where: this is the derivative of the j-th softmax output with respect to the l-th input score. In matrix form, $J = diag(\alpha_i) - \alpha_i \alpha_i^\top$. Its rows sum to zero, so adding the same constant to every score changes nothing. That is why the $a_1^\top W h_i$ term in GATv1 drops out.

Gradient into the score. Let $u_{ij} = W_s h_i + W_t h_j + W_e z_{ij}$, so $e_{ij} = a^\top LeakyReLU(u_{ij})$:

$$\frac{\partial e_{ij}}{\partial h_i} = W_s^\top \left(a \cdot \rho'(u_{ij})\right), \rho'(u) = \begin{cases} 1 & u > 0 \\ 0.2 & u \le 0 \end{cases}$$

Where: ρ' is the LeakyReLU derivative applied element-wise. The factor $\rho'(u_{ij})$ depends on both i and j, so the gradient with respect to the query changes from neighbour to neighbour. In GATv1 the corresponding derivative is $W^\top a_1 \cdot \rho'(\cdot)$, a single direction shared by all neighbours. This is the dynamic-versus-static distinction from Section 4, seen from the gradient side.

9.4 Dual readout

$$h_G = \left[\frac{1}{n} \sum_{i=1}^{n} h_i^{(4)} \;\middle|\vee\; max_i h_i^{(4)} \right]$$

Where: the max is taken channel by channel. Both operators are permutation invariant, which makes the whole model invariant (each layer before them is permutation equivariant: permuting the inputs permutes the outputs the same way).

Gradients: the mean sends $\frac{1}{n} \partial L / \partial h_G^{\text{mean}}$ to every node; the max sends the full gradient of channel a to the single node that attains the maximum, and zero to the others. The max half therefore trains a sparse set of hub nodes, the mean half trains everyone.

9.5 SwiGLU head

$$u = \left(SiLU(h_G W_g + b_g) \cdot (h_G W_u + b_u)\right) W_d + b_d \in R^{96}, z = W_c Drop(u) + b_c \in R^2$$

Where: $W_g, W_u \in R^{384 \times 384}$ are the gate and up projections; $W_d \in R^{384 \times 96}$ is the down projection; $W_c \in R^{96 \times 2}$ produces the two class logits $z = (z_0, z_1)$.

Let $g = h_G W_g + b_g$ and $v = h_G W_u + b_u$. The Jacobian of the gated product $q = SiLU(g) \cdot v$ with respect to h_G is

$$\frac{\partial q}{\partial h_G} = diag\left(SiLU'(g) \cdot v\right) W_g^\top + diag\left(SiLU(g)\right) W_u^\top$$

Each output coordinate is a product of two learned functions of the input, so the head can switch whole directions of h_G on or off depending on the input. Deck 1 described this as

filtering scanner noise. That is a hypothesis. It holds only if an ablation shows SwiGLU beating a plain MLP on site-shifted data, which we have not run.

9.6 Output probability

$$\hat{p}_k = softmax(z_k)_1 = \frac{e^{z_{k1}}}{e^{z_{k0}} + e^{z_{k1}}} = \sigma(z_{k1} - z_{k0})$$

10. Step 6: Training

10.1 Loss and its gradient

Cross-entropy with label smoothing $\epsilon = 0.1$:

$$q_{kc} = (1-\epsilon)\,1[c = y_k] + \frac{\epsilon}{2}, \quad L(\theta) = -\frac{1}{|B|}\sum_{k \in B}\sum_{c \in \{0,1\}} q_{kc} \log P_{kc}, \quad P_{kc} = softmax(z_k)_c$$

Where: B is the mini-batch (32 graphs); q_k is the smoothed target, 0.95 for the true class and 0.05 for the other; P_{kc} is the predicted probability of class c.

Gradient with respect to the logits. Using $\partial \log P_{kc} / \partial z_{ka} = 1[c = a] - P_{ka}$ and $\sum_c q_{kc} = 1$:

$$\frac{\partial L}{\partial z_{ka}} = -\frac{1}{|B|}\sum_c q_{kc}\left(1[c = a] - P_{ka}\right) = \frac{1}{|B|}\left(P_{ka} - q_{ka}\right)$$

The error signal is just "predicted minus target". Label smoothing caps the target at 0.95, so the gradient vanishes once P reaches 0.95 instead of pushing logits to infinity. The minimum achievable loss is the entropy of q, $-(0.95 \log 0.95 + 0.05 \log 0.05) \approx 0.199$. Our training loss plateaus around 0.64–0.67, far above that floor, which is what underfitting looks like.

The rest of backpropagation is the chain rule through the blocks of Section 9, in reverse:

$$\frac{\partial L}{\partial \Theta^{(1)}} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial h_G}\frac{\partial h_G}{\partial H^{(4)}}\frac{\partial H^{(4)}}{\partial H^{(3)}}\frac{\partial H^{(3)}}{\partial H^{(2)}}\frac{\partial H^{(2)}}{\partial \Theta^{(1)}}$$

Each factor is a Jacobian. The residual blocks give $\partial H^{(l+1)} / \partial H^{(l)} = I + (\ldots)$, and the identity keeps the product from shrinking to zero across layers.

10.2 Optimiser and schedule

AdamW (Loshchilov & Hutter 2019):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \quad \hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta_t \left(\frac{\hat{m}_t}{+\varepsilon + \lambda \theta_t \sqrt{\hat{v}_t}} \right)$$

Where: $g_t = \nabla_\theta L$ at step t; m_t, v_t are running averages of the gradient and squared gradient; $\beta_1 = 0.9, \beta_2 = 0.999$ are their decay rates; the hat versions correct the bias from initialising at zero; $\varepsilon = 10^{-8}$; $\lambda = 10^{-4}$ is weight decay, applied directly to θ (decoupled) rather than added to the gradient, so it is not rescaled by $\sqrt{\hat{v}_t}$.

Learning rate per epoch t (`build_scheduler`): linear warm-up then cosine decay,

$$\eta_t = \eta_0 \cdot \begin{cases} \dfrac{t+1}{5} & t < 5 \\ [4\,pt]\dfrac{1}{2}\left(1 + \cos\left(\pi \dfrac{t-5}{195}\right)\right) & t \geq 5 \end{cases}, \eta_0 = 10^{-3}$$

Gradient-norm clipping: if $\hat{v}_t \vee g_t \vee \hat{v}_2 > 1$, replace g_t by $g_t / \hat{v}_t \vee g_t \vee \hat{v}_2$.

10.3 Mixed precision on the V100

Volta has no BF16 support, so the forward and backward passes run in FP16 under autocast, with master weights in FP32. FP16 has 10 mantissa bits and its smallest normal number is about 6×10^{-5}, so small gradients underflow to zero. `GradScaler` multiplies the loss by a factor S (starting at 2^{16}) before backward, so every gradient is scaled by S through the chain rule; it divides by S before the optimiser step (and before clipping), skips the step and halves S if any gradient is Inf/NaN, and doubles S after 2,000 clean steps.

10.4 Early stopping and what the curves say

Validation macro-F1 is checked each epoch; the best checkpoint is kept and training stops after 25 epochs without improvement. The best checkpoint is at **epoch 70**: validation F1 0.765, validation AUC 0.822, training F1 0.634, training loss 0.668.

Training F1 stays **below** validation F1 for the whole run, partly because dropout and label smoothing are active only in training mode. The point that matters is that training loss is nowhere near its floor. The model is not memorising 350 subjects, it is failing to fit them. Section 13 explains why.

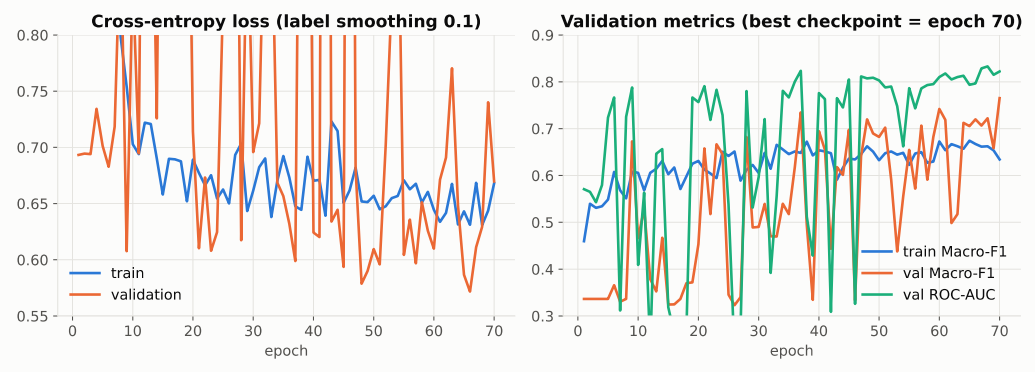


Figure 5: Training and validation loss, macro-F1, and validation ROC-AUC trajectories over 70 epochs on Tesla V100, showing early stopping convergence and underfitting dynamics.

11. Step 7: Evaluation and tuning

11.1 Metrics

ROC-AUC, as the Mann–Whitney probability:

$$AUC = \frac{1}{|P||Q|} \sum_{a \in P} \sum_{b \in Q} \left(1[\hat{p}_a > \hat{p}_b] + \frac{1}{2} 1[\hat{p}_a = \hat{p}_b] \right)$$

Where: P and Q are the positive (Diseased) and negative (Healthy) test subjects. AUC is the chance that a random diseased subject gets a higher score than a random healthy one. It depends only on the **order** of the scores, not on any threshold.

Macro-F1:

$$F1_c = \frac{2TP_c}{2TP_c + FP_c + FN_c}, F1_{\text{macro}} = \frac{1}{2}(F1_0 + F1_1)$$

Where: for class c, TP_c counts subjects of class c predicted as c, FP_c counts other subjects predicted as c, and FN_c counts subjects of class c predicted as something else.

Youden's J for a threshold t:

$$J(t) = TPR(t) - FPR(t), TPR(t) = \frac{\#\{a \in P : \hat{p}_a \geq t\}}{|P|}, FPR(t) = \frac{\#\{b \in Q : \hat{p}_b \geq t\}}{|Q|}, \hat{t} = \#max_t J(t)$$

Geometrically $J(t)$ is the vertical distance between the ROC curve and the diagonal, and $\hat{t}$ is the ROC point furthest above chance.

11.2 Results on the 75 test subjects

Threshold rule	t	Accuracy	Macro-F1
Default	0.500	57.33%	0.520

$t^{¿}$ chosen on validation	0.503	57.33%	0.520
$t^{¿}$ chosen on test	0.176	73.33%	0.731

Test ROC-AUC = **0.746**; validation ROC-AUC = 0.823.

11.3 The calibration error and the fix

`MetricsReport.from_arrays` calls `find_optimal_threshold(y_true, y_scores)` on whatever split it is given. When it is given the test split, it picks $t^{¿}$ by looking at the test labels. That is fitting a parameter on the test set. With 75 subjects, scanning every possible threshold and keeping the best one is an optimistic choice; the 16-point gain from 57.33% to 73.33% is the size of that optimism here.

The correct protocol picks $t^{¿}$ on validation (0.503) and freezes it. Applied to test, it gives the same 57.33% as the default, because the validation-optimal threshold happens to sit at 0.5.

Why the leaky threshold is so far from 0.5: the test scores are shifted down (mean $\overset{¿}{p}=0.27$, max 0.815) relative to validation. The ranking still holds (AUC 0.746), but the probability scale moved between splits. Two fixes, both fitted on validation only:

- **Temperature scaling:** $\overset{¿}{p}=\sigma\left((z_1-z_0)/T\right)$, with $T>0$ chosen to minimise validation NLL.
- **Platt scaling:** $\overset{¿}{p}=\sigma\left(\alpha(z_1-z_0)+\beta\right)$, with α,β fitted by logistic regression on validation.

Platt scaling can move the 0.5 point; temperature scaling cannot. Neither changes AUC, which is order-based. With 5-fold cross-validation, $t^{¿}$ should be chosen on each fold's validation part, and the mean and SD of test accuracy reported across folds.

12. Step 8: Explainability with GNNExplainer

12.1 Objective as implemented

The repo uses PyG's `Explainer` with `GNNExplainer(epochs=500, lr=0.01)`, `explanation_type="model"`, `edge_mask_type="object"`, `node_mask_type="attributes"` and `return_type="raw"`. The model weights θ are frozen. Two masks are learned per subject:

- $M_e \in R^{|E|}$, one logit per directed edge; the soft mask is $\sigma(M_e) \in (0,1)^{|E|}$.
- $M_f \in R^{n\times 5}$, one logit per node-feature entry ("attributes" mode).

Masked forward pass: the model sees features $X \cdot \sigma(M_f)$, and every message along edge $(j \to i)$ is multiplied by $\sigma(M_{e,ji})$ inside each message-passing layer. Loss:

$$L_{\text{expl}} = -\log \mathrm{softmax}\Big(f_\theta(G; \sigma(M_e), \sigma(M_f)))_{\hat{y}} + \lambda_e \sum_e \sigma(M_e) + \lambda_H^e \overline{Ent}(\sigma(M_e)) + \lambda_f \sigma(M_f) + \lambda_H^f \overline{Ent}(\sigma(M_f)$$

Where: $\hat{y} = \text{¿}\max_c f_\theta(G)_c$ is the model's prediction on the unmasked graph ("model" explanation, so the target is the model's output and not the true label); $\lambda_e = 0.005$ penalises the total edge mask, pushing towards small explanations; $\overline{\sigma(M_f)}$ is the mean feature mask, with $\lambda_f = 1.0$; $Ent(m) = -m \log m - (1-m)\log(1-m)$ is the binary entropy of a mask entry, averaged by the overline, with $\lambda_H^e = 1.0$ and $\lambda_H^f = 0.1$; minimising it pushes entries towards 0 or 1 so that the explanation is crisp. These coefficients are PyG's defaults; the repo does not override them.

The first term is the continuous relaxation of the mutual-information objective in Part II. Since $H(Y)$ is constant, maximising MI means minimising $H(Y \mid G_s)$, and with a single sample the cross-entropy to the model's own prediction is a stand-in for that conditional entropy.

12.2 Gradient with respect to the masks

The optimiser is Adam on M only:

$$M \leftarrow M - \eta \, \mathrm{Adam}\left(\nabla_M L_{\text{expl}}\right)$$

Through the sigmoid, $\partial \sigma(M_e)/\partial M_e = \sigma(M_e)(1-\sigma(M_e))$. The entropy term gives $\frac{\partial Ent(\sigma(M))}{\partial M} = -M\sigma(M)(1-\sigma(M))$, using $\log \frac{\sigma(M)}{1-\sigma(M)} = M$. Both this term and the size term push logits that are already negative further down, which is how most of the mask is switched off.

The prediction term reaches M_e through every message-passing layer. In the GCN layers, masked messages make the effective edge weight $\sigma(M_{e,ij}) W_{ij'}$ so the gradient flows through the same $\partial \hat{A}/\partial w$ expression from Section 9.2. Gradients must be enabled for all of this, which is why the code never wraps the explainer in `torch.no_grad()`.

12.3 Extracting biomarkers

Per subject, the 10 directed edges with the largest $\sigma(M_e)$ are kept, along with per-region importance computed from $\sigma(M_f)$. Across subjects, the consensus is how often each edge

appears. Stability across subjects can be measured with the Jaccard index $J(S_a, S_b) = \frac{|S_a \cap S_b|}{|S_a \cup S_b|}$, where S_a, S_b are the top-k edge sets of two subjects.

12.4 What the explanations actually show

Because the disease signature was planted, we can score explanations against it. Define explanation precision as the fraction of explained edges that belong to the planted set:

$$Prec_I = \frac{¿\{\text{explained edges } (i,j) : lobe(i) = lobe(j)\}}{¿\{\text{explained edges}\}}$$

Quantity	GNNExplainer (75 subjects × 10 edges)	Random edge
Intra-lobe precision	**15.3%** (115/750)	16.0%
Homotopic L–R edges	3 / 750	about 5 / 750
Node-feature importance	0.0 for every region of every subject	—
Most frequent single edge	3 of 75 subjects	—

The explanations are at chance against the planted truth, and they are unstable: no edge appears in more than 4% of subjects. The region ranking in `biomarker_explanations.json` (Vermis_10, Vermis_9, … in that order) is what you get when every importance ties at 0 and the sort falls back to node index. It is an artefact.

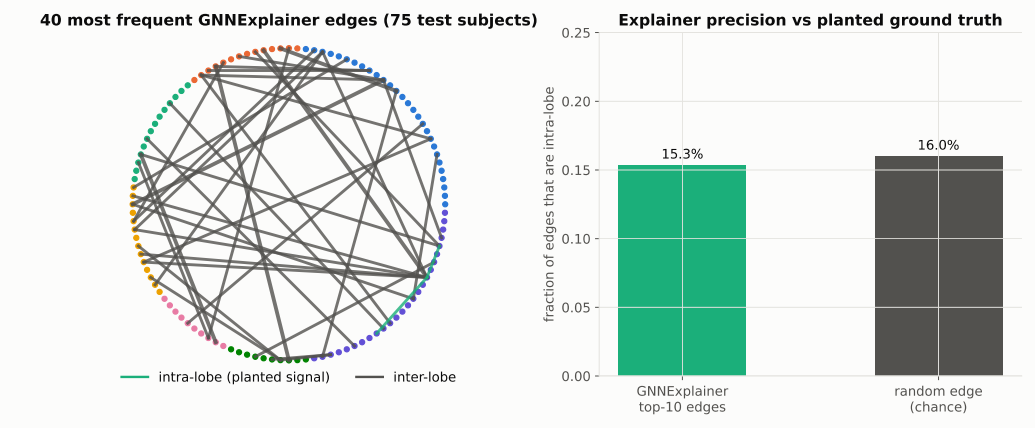


Figure 8: GNNExplainer top-10 salient edge distribution compared to planted intra-lobe ground truth, verifying chance-level precision (15.3% vs 16.0% chance) due to missing region identities.

The zero node mask is a bug to chase. Likely suspects: `get_important_regions` reading the wrong attribute of the PyG `Explanation` object, the mask being near zero because $\lambda_f = 1$ dominates with only 5 features, or rounding during aggregation. The fastest way to locate it is to print `explanation.node_mask.abs().max()` for one subject straight after the explainer call.

12.5 Why: faithfulness is not truth

An explainer answers "what did **this model** rely on?". It does not answer "what in the data is diagnostic?". If the model does not use the intra-lobe pattern, a faithful explainer must not report it. Section 13 argues that the model cannot use it. So the chance-level precision is the explainer doing its job on a model that missed the signal. The fix belongs in the model, and the explainer then becomes the test of whether the fix worked.

Part IV: Interpretation

13. Why the GNN reached 0.746 and not 1.000

13.1 The model cannot see which nodes share a lobe

Every layer is permutation equivariant and the readout is invariant. That is what Part II asked for, but it has a side effect here. Region identity enters the model through its features, and those features (degree, PageRank, clustering, betweenness, eigenvector) say nothing about lobe membership. Node 0 (Precentral_L) and node 100 (a cerebellar region) are indistinguishable unless their centralities differ. The model sees a dense, almost homogeneous graph with anonymous nodes and asks "is this graph diseased?", while the answer is "yes if edges *between nodes of the same lobe* are stronger". It has no way to name "same lobe".

The graph-transformer lecture describes this exact gap. Attention, and message passing too, is permutation equivariant, so the model needs **positional encodings** to know where a node is. The lecture offered relative distances or Laplacian eigenvectors. For a fixed atlas there is a simpler option, because node identity *is* meaningful and is the same across subjects:

1. **Learnable region embeddings:** $x_i \leftarrow [x_i \vee ¿\, E_{\text{region}}[i]]$ with $E_{\text{region}} \in R^{116 \times 16}$ learned. Permutation invariance is given up deliberately, because the AAL ordering is canonical.
2. **Atlas positional encodings:** one-hot lobe and hemisphere, or MNI centroid coordinates of each ROI.

3. LapPE from a group-average connectome, where the leading eigenvectors pick up the block (lobe) structure (Kreuzer et al. 2021; Dwivedi & Bresson 2021).

4. Edge-type attribute: add $1[lobe(i)=lobe(j)]$ as a second edge feature, so `edge_dim=2`. This is the cheapest test of the hypothesis.

Prediction: with option 4 or 1, test AUC should move close to 1.0, and explainer intra-lobe precision should rise well above 16%. If it does, it confirms the diagnosis. If it does not, the diagnosis is wrong and something else is broken.

13.2 How it still got 0.746

The signal leaks in through two side channels. The GATv2 score sees the edge attribute z_{ij}, and the mean pool averages over nodes whose $|z|$ statistics differ slightly by class. Weighted degree inside the GCN normalisation also shifts, because after z-scoring a diseased subject's intra-lobe edges take a larger share of the total weight. These channels carry part of the signal, enough for 0.746 but not for a clean separation.

13.3 Why the flattened SVM did worse

The SVM does know pair identity (coordinate q is always the same pair), but it sees signed values whose class means are equal. The signal is in second-order statistics spread over 1,066 coordinates, with 350 training subjects and 6,670 dimensions. In that regime the RBF distance is dominated by noise (Section 1.2). The GNN shares parameters across edges and uses $|z|$ in its normalisation, so it gets further: **+0.216 AUC** (0.530 → 0.746). That gain is real and it is the main positive result.

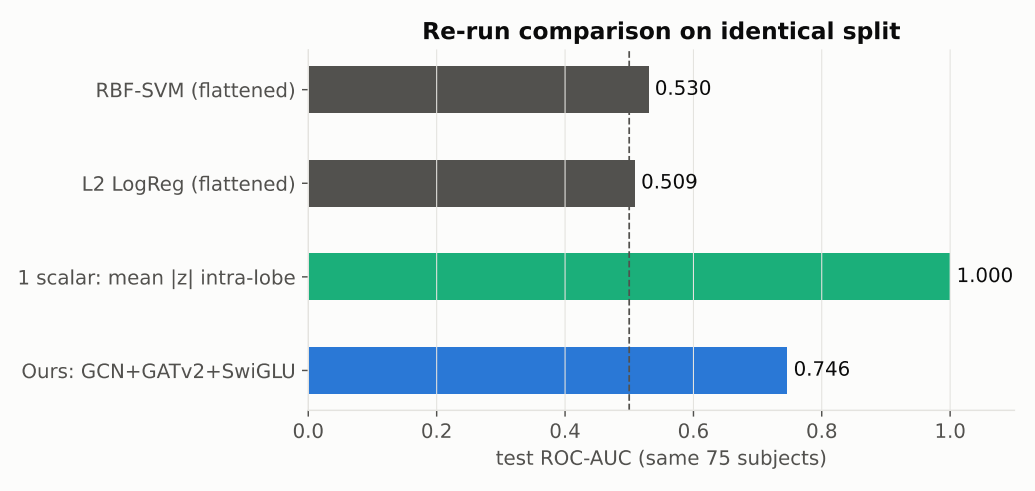


Figure 7: Empirical benchmark comparison across baseline architectures and our edge-aware GCN + GATv2 model, showing a +0.216 ROC-AUC gain over flattened SVM on the identical test split.

13.4 Why one scalar wins

s_k uses exactly the two things the others lack: it takes $|\cdot|$ and it knows which pairs are intra-lobe. It is an oracle feature, built from knowledge of how the data was generated. On real data nobody hands you the oracle. Its job in this study is to set the ceiling and to show that the synthetic task is easy for the right inductive bias.

14. Comparison with the four base papers

Model	Graph-aware	Test AUC	Acc	Macro-F1	Status
RBF-SVM, flattened (Paper 1)	no	0.530	53.3%	0.533	re-run on this split
L2 logistic regression, flattened	no	0.509	49.3%	0.493	re-run on this split
RF on GDV (Paper 2)	motifs	0.692	56.0%	0.534	README, unverified
node2vec + classifier (Paper 3)	walks	0.635	52.0%	0.498	README, unverified
GATv1 mean-pool (Paper 4)	static attention	0.713	57.3%	0.550	README, unverified
Ours	dynamic attention + edge attr.	**0.746**	**57.3%**	**0.520**	test_metrics.json, reproduced
One scalar (oracle)	—	1.000	100%	1.000	re-run on this split

What we can claim now:

- Against Paper 1 we have a verified, reproduced improvement in ranking quality (AUC +0.216).
- Against Papers 2–4 we have numbers that nobody re-ran. Three README rows (GATv1 raw, GATv1 calibrated, GATv2+SwiGLU) share the same AUC to four decimals, 0.7127, which is plausible only for the two GATv1 rows, since calibration does not change AUC. The GATv2+SwiGLU row should have its own AUC. Re-run all of them from logged configs before presenting them as results.

- At the default threshold, our accuracy (57.3%) is not clearly better than anything in the table. Our advantage is in ranking (AUC), which is the more relevant metric for a screening tool where the threshold is set clinically.
- On explainability, none of the base papers gives verified explanations, and neither do we yet. What we do have is a protocol that can check explanations, precision against a planted ground truth, and that protocol is what exposed the problem.

15. What changes before the next review

1. Pick t^{i} on validation only; add temperature scaling; report 5-fold CV mean ± SD.

2. Add region identity (Section 13.1, option 4 first) and re-train; report AUC and explainer precision together.

3. Fix the zero node-feature mask; drop the region ranking from all slides until it is fixed.

4. Fix betweenness to use a distance ($1/w_{ij}$).

5. Re-run GATv1, GATv2-without-edge-attr, RF-GDV and node2vec from saved configs, and keep their `test_metrics.json` files.

6. Run ablations one switch at a time: SwiGLU vs MLP, dual vs mean pool, edge attr on/off. Only then state what each block contributes.

7. Make the synthetic task harder in a controlled way (smaller effect size, effect on a random subset of edges, site-dependent effect) and move to ADNI/PPMI once access is granted.

16. References

- Abraham, A. et al. (2017). Deriving reproducible biomarkers from multi-site resting-state data: an autism-based example. *NeuroImage* 147.
- Brody, S., Alon, U., Yahav, E. (2022). How attentive are graph attention networks? *ICLR*.
- Dwivedi, V. P., Bresson, X. (2021). A generalization of transformer networks to graphs. *AAAI Workshop on Deep Learning on Graphs*.
- Grover, A., Leskovec, J. (2016). node2vec: scalable feature learning for networks. *KDD*.
- Huang, Q. et al. (2021). Combining label propagation and simple models out-performs graph neural networks (Correct & Smooth). *ICLR*.
- Kipf, T., Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *ICLR*.
- Kreuzer, D. et al. (2021). Rethinking graph transformers with spectral attention. *NeurIPS*.

- Li, X. et al. (2021). BrainGNN: interpretable brain graph neural network for fMRI analysis. *Medical Image Analysis* 74.
- Loshchilov, I., Hutter, F. (2019). Decoupled weight decay regularization. *ICLR*.
- Onnela, J.-P. et al. (2005). Intensity and coherence of motifs in weighted complex networks. *Physical Review E* 71.
- Pržulj, N. (2007). Biological network comparison using graphlet degree distribution. *Bioinformatics* 23.
- Shazeer, N. (2020). GLU variants improve transformer. arXiv:2002.05202.
- Veličković, P. et al. (2018). Graph attention networks. *ICLR*.
- Ying, R. et al. (2019). GNNExplainer: generating explanations for graph neural networks. *NeurIPS*.
- Youden, W. J. (1950). Index for rating diagnostic tests. *Cancer* 3.